КЕЙС 3

Приложение для оплаты чеков по QR «Онлайн-чаевые»

Сеть ресторанов «Вкус-Сервис»

Команда «Дверь»

Документ для защиты проекта

2026

2. Оглавление

1. Титульный лист
2. Оглавление
3. Требования (FR / NFR)
4. Роли и права
5. Правила QR и платежа
6. Словарь данных (Data Dictionary)
7. ER-модель данных
8. Генерация QR и диалог выбора чаевых
9. Эмулятор платежа
10. Отчётность
11. Спецификация API
12. Информационная безопасность
13. Приложение (диаграммы, тесты, артефакты)
14. Исходный код прототипа (C++)

3. Требования

3.1. Функциональные требования (FR)

ID	Требование	Приоритет	Тест
FR-1	Сгенерировать QR на чек (merchant_id, employee_id, receipt_id, nonce, exp, signature)	Must	Т3-1
FR-2	Зарегистрировать мерчанта: заявка, статус PENDING, активация или блокировка администратором	Must	ручная
FR-3	Зарегистрировать сотрудника по приглашению, хранить только маску реквизитов (****NNNN)	Must	ручная
FR-4	Проверять QR при открытии: HMAC, exp, nonce, статус мерчанта, receipt_id	Must	Т3-3, Т3-4
FR-5	Показать диалог выбора чаевых (4 экрана) и рассчитать сумму	Must	Т3-1
FR-6	Эмулировать платёж и вернуть SUCCESS / DECLINED / PENDING	Must	Т3-1, Т3-2
FR-7	Идемпотентность: повтор с тем же idempotency_key возвращает существующий платёж	Must	Т3-4
FR-8	Отменить платёж до подтверждения; запрет отмены из PENDING и SUCCESS	Must	Т3-5
FR-9	Записывать каждый переход статуса платежа в журнал аудита	Must	Т3-1…Т3-5
FR-10	Формировать отчёты по чаевым: по мерчанту, официанту, по дням	Should	сверка CSV

3.2. Нефункциональные требования (NFR)

ID	Требование
NFR-1	QR-подпись проверяется за ≤ 50 мс
NFR-2	Срок действия QR (TTL) 24 часа, после — QR_EXPIRED
NFR-3	Подпись QR: HMAC-SHA256; ключ вне кода и Git; безопасное сравнение
NFR-4	payment_details только маска ****NNNN; PAN/CVV не хранятся; в логах маскируются
NFR-5	Rate limit: не более 10 запросов в минуту с одного IP на проверку QR и создание платежа
NFR-6	Аудит: каждое изменение статуса записывается (8 полей), записи неизменяемы

3.3. Ограничения

- Прототип консольный (C++), без веб-интерфейса и реальных платёжных систем.
- Платёж эмулируется: реальные деньги не списываются.
- Данные в CSV; исходные файлы не изменяются, результаты — в отдельные файлы.

3.4. Пользовательские сценарии

Сценарий 1. Клиент оставил чаевые

1. Клиент сканирует QR → система проверяет и показывает имя официанта, сумму, названиезаведения.
2. Клиент выбирает % чаевых, видит итог, нажимает «Подтвердить».
3. Эмулятор возвращает SUCCESS, чаевые начислены, событие в аудите.

Сценарий 2. Клиент отменил платёж

Клиент на экране подтверждения нажимает «Отмена»: статус CREATED → CANCELED, деньги не списываются.

Сценарий 3. QR устарел
Клиент сканирует QR через сутки → QR_EXPIRED (410). Официант выдаёт новый QR.

4. Роли и права

4.1. Матрица ролей

Действие	Мерчант	Официант	Клиент	Администратор
Регистрация в платформе	да	да (по приглашению)	нет (анонимен)	да
Генерация QR на чек	да	да	нет	нет
Просмотр своих чаевых	да	да (только свои)	нет	да
Сканирование QR	нет	нет	да (своего чека)	нет
Оплата чаевых	нет	нет	да	нет
Просмотр отчёта по заведению	да	нет	нет	да
Блокировка мерчанта	нет	нет	нет	да
Изменение справочников	да (свои сотрудники)	нет	нет	да

4.2. Процесс регистрации мерчанта

Шаг	Кто	Что происходит	Проверки
1	Мерчант	Подаёт заявку: название, описание, контакты	Название 1–100 символов; описание ≤200; уникальность названия
2	Система	Создаёт merchant_id (m_NNN), статус PENDING	merchant_id уникален; статус только PENDING
3	Администратор	Решает: ACTIVE или BLOCKED	Только администратор; мерчант сам себя не активирует
4	Система	Сохраняет статус, аудит, уведомление	При ACTIVE — можно добавлять сотрудников и QR; иначе MERCHANT_BLOCKED

4.3. Процесс регистрации сотрудника

Шаг	Кто	Что происходит	Проверки
1	Мерчант	Приглашает официанта, создаёт одноразовую ссылку	Мерчант ACTIVE; приглашение ограничено по времени
2	Официант	Переходит по ссылке, подтверждает имя, вводит реквизиты	Приглашение действительно; формат реквизитов корректен
3	Система	Создаёт employee_id (e_NNN), сохраняет только маску ****NNNN	Полный номер не сохраняется; один официант — один мерчант
4	Система	Помечает приглашение использованным, аудит, уведомление	Повторное использование запрещено

5. Правила QR и платежа

5.1. Проверки QR

ID	Проверка	Действие	Код ошибки
Q-1	signature совпадает с HMAC	продолжить	—

Q-2	exp ≥ now	продолжить	QR_EXPIRED
Q-3	nonce ещё не использован	продолжить	QR_USED
Q-4	merchant.status = ACTIVE	продолжить	MERCHANT_BLOCKED
Q-5	receipt_id в запросе = receipt_id в QR	продолжить	RECEIPT_MISMATCH
Q-6	tip_pct ≤ 50	продолжить	TIP_OUT_OF_RANGE

5.2. Правила платежа

ID	Условие	Действие	Код ошибки
P-1	base_amount ≤ 0 или > 1 000 000	отклонить	AMOUNT_OUT_OF_RANGE
P-2	повтор idempotency_key	вернуть существующий платёж (HTTP 200)	— (не ошибка)
P-3	amount_total > 1 200 000	отклонить	AMOUNT_OUT_OF_RANGE
P-4	отмена в статусе PENDING или SUCCESS	отклонить	CANCEL_NOT_ALLOWED

5.3. Обоснование правил

ID	Какую проблему закрывает
Q-1	Подделка QR: без секретного ключа нельзя посчитать верную подпись
Q-2	Использование старого QR: срок жизни ограничен 24 часами
Q-3	Повторное использование QR (replay): один nonce работает один раз
Q-4	Платежи заблокированному или не прошедшему проверку заведению
Q-5	Подмена чека: QR привязан к конкретному receipt_id
Q-6	Ошибки и злоупотребления с суммой чаевых: потолок 50%
P-1	Нулевые, отрицательные и абсурдно большие суммы
P-2	Двойное списание при повторной отправке (сбой сети, двойной клик)
P-3	Слишком большой итог (чек + чаевые)
P-4	Отмена платежа, который уже ушёл в шлюз или завершён

6. Словарь данных (Data Dictionary)

6.1. merchants

Поле	Тип	Обяз.	Ограничение	Пример	Код ошибки
merchant_id	string	да	m_NNN	m_001	—
name	string	да	1–100 символов	Вкус-Сервис Тверская	—
status	enum	да	ACTIVE/BLOCKED/PENDING	ACTIVE	MERCHANT_BLOCKED
description	string	нет	0–200	Ресторан сети	—

6.2. employees

Поле	Тип	Обяз.	Ограничение	Пример	Код ошибки
employee_id	string	да	e_NNN, уникален	e_001	—
merchant_id	string	да	m_NNN, FK → merchants	m_001	MERCHANT_NOT_FOUND
name	string	да	1–100 символов	Иван Петров	—
payment_details	string	да	только маска ****NNNN	****1234	INVALID_PAYMENT_DETAILS

6.3. payments

Поле	Тип	Обяз.	Ограничение	Пример	Код ошибки
payment_id	string	да	p_NNNN	p_0001	—
amount	decimal	да	> 0, ≤ 1 000 000	637.00	AMOUNT_OUT_OF_RANGE
tip	decimal	да	≥ 0, ≤ amount × 0.5	63.70	TIP_OUT_OF_RANGE
status	enum	да	CREATED/PENDING/SUCCESS/DECLINED/CANCELED	SUCCESS	—
merchant_id	string	да	m_NNN	m_001	MERCHANT_NOT_FOUND
employee_id	string	да	e_NNN	e_001	EMPLOYEE_NOT_FOUND
timestamp	datetime	да	ISO 8601	2025-11-01T18:00:00Z	—

6.4. qr_links

Поле	Тип	Обяз.	Ограничение	Пример	Код ошибки
qr_id	string	да	qr_NNN, уникален	qr_001	—
merchant_id	string	да	m_NNN, FK	m_001	MERCHANT_NOT_FOUND
employee_id	string	да	e_NNN, FK	e_001	EMPLOYEE_NOT_FOUND
receipt_id	string	да	r_NNNN, уникален для мерчанта	r_1001	—
nonce	string	да	6 hex, одноразовый	c82561	QR_USED
exp	datetime	да	ISO 8601 UTC, > now	2025-11-01T23:59:00Z	QR_EXPIRED

Поле	Тип	Обяз.	Ограничение	Пример	Код ошибки
signature	string	да	HMAC-SHA256, 16 hex	68d0f361ac38c12b	INVALID_QR

7. ER-модель данных

7.1. Сущности

Сущность	Что хранит	Зачем
Merchant	merchant_id, name, status, description	справочник мерчантов
Employee	employee_id, merchant_id, name, payment_details	официанты, получатели чаевых
QR	qr_id, merchant_id, employee_id, receipt_id, nonce, exp, signature	одноразовая подписанная ссылка
Payment	payment_id, amount, tip, status, timestamp, …	факт оплаты: чек + чаевые
Report	report_id, merchant_id, period, tips_total	агрегаты для отчётности
AuditEvent	event_id, payment_id, qr_id, action, from/to_status, actor, at	журнал событий для разбора

7.2. Связи

Связь	Тип	Обоснование
Merchant → Employee	1:N	у ресторана много официантов, официант — в одном ресторане
Merchant → QR	1:N	ресторан создаёт много QR
Employee → QR	1:N	у официанта много QR (по одному на чек)
QR → Payment	1:1	один QR — одна успешная оплата (nonce одноразовый)
Merchant → Report	1:N	по одному отчёту за каждый месяц
Payment → AuditEvent	1:N	каждый переход статуса — отдельное событие

7.3. Диаграмма ER

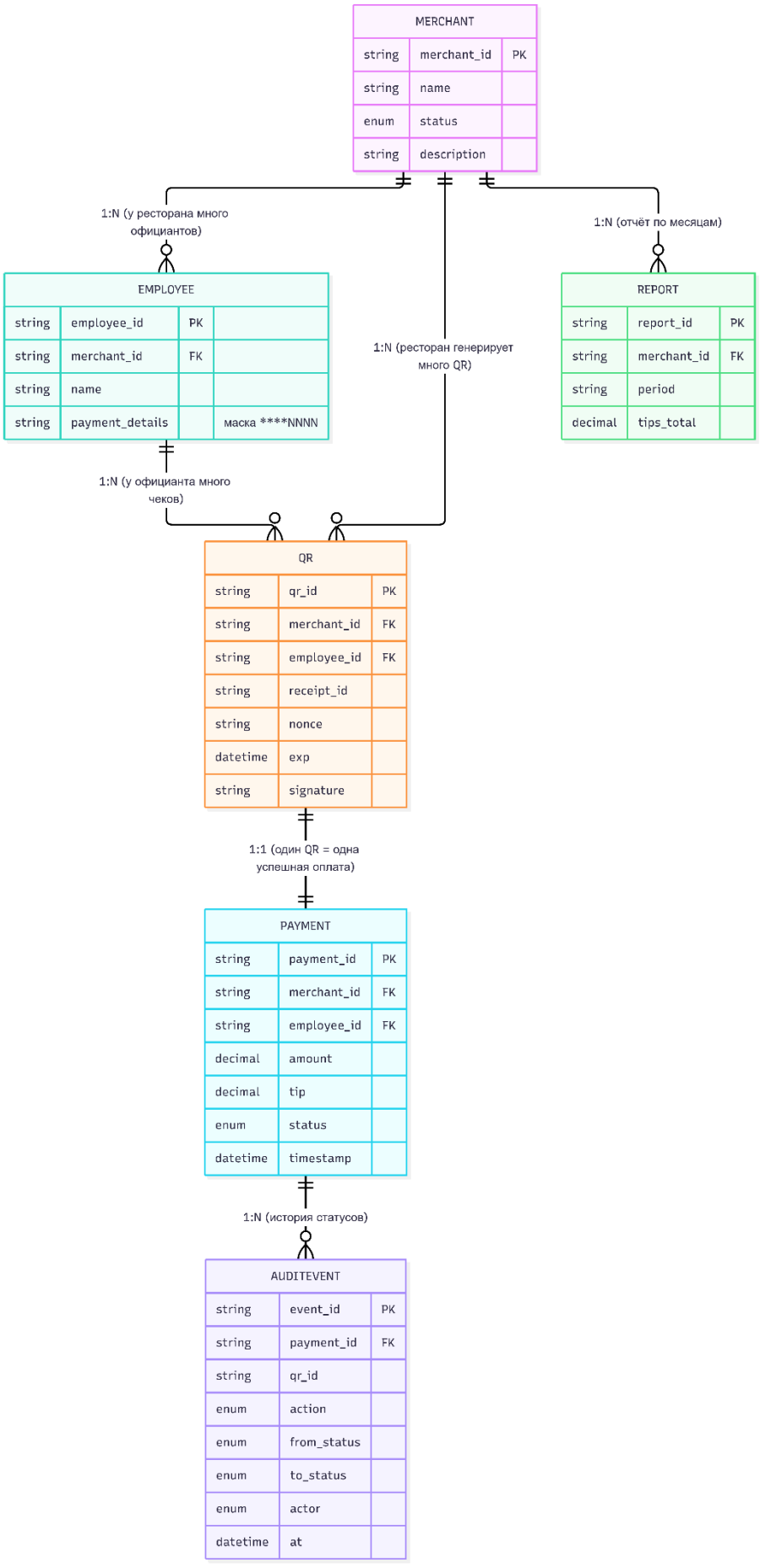

MERCHANT
string merchant_id PK
string name
enum status
string description
1:N (у ресторана много официантов)
1:N (отчёт по месяцам)
EMPLOYEE
string employee_id PK
string merchant_id FK
string name
string payment_details маска ****NNNN
REPORT
string report_id PK
string merchant_id FK
string period
decimal tips_total
1:N (ресторан генерирует много QR)
1:N (у официанта много чеков)
QR
string qr_id PK
string merchant_id FK
string employee_id FK
string receipt_id
string nonce
datetime exp
string signature
1:1 (один QR = одна успешная оплата)
PAYMENT
string payment_id PK
string merchant_id FK
string employee_id FK
decimal amount
decimal tip
enum status
datetime timestamp
1:N (история статусов)
AUDITEVENT
string event_id PK
string payment_id FK
string qr_id
enum action
enum from_status
enum to_status
enum actor
datetime at

Рисунок 1. ER-диаграмма (Merchant, Employee, QR, Payment, Report, AuditEvent)

8. Генерация QR и диалог выбора чаевых

8.1. Payload QR

Поле	Источник	Пример	Зачем
merchant_id	merchants.csv	m_001	идентификатор заведения и его статус
employee_id	employees.csv	e_001	кому пойдут чаевые
receipt_id	сгенерирован	r_1001	привязка к чеку, уникален внутри мерчанта
nonce	random 6 hex	c82561	защита от повторного использования
exp	+24 ч от now	2025-11-01T23:59:00Z	срок действия (QR_EXPIRED)
signature	HMAC-SHA256	68d0f361ac38c12b	защита от подделки

8.2. Подпись HMAC-SHA256

Подпись считается по пяти полям (все, кроме самой подписи):

```
payload = f"{merchant_id}|{employee_id}|{receipt_id}|{nonce}|{exp}" sig =
hmac.new(SECRET, payload.encode(), hashlib.sha256).hexdigest()[:16]
```

Пример (qr_001): payload = m_001|e_001|r_1001|c82561|2025-11-01T23:59:00Z → signature = 68d0f361ac38c12b

8.3. Диалоговая форма (4 экрана)

Экран 1. Приветствие — имя официанта, сумма чека, название заведения. Проверка QR перед показом.

Экран 2. Выбор процента — 0 / 5 / 10 / 15 / 20 % или своя сумма (≤ 50%). Показ итога.

Экран 3. Подтверждение — сумма чека, чаевые, итог. Кнопки «Подтвердить» и «Отмена».

Экран 4. Результат — Успех / Отказ / Таймаут / Отмена.

9. Эмулятор платежа

9.1. Состояния платежа

- CREATED — создан после проверки QR, клиент ещё не подтвердил.
- PENDING — клиент подтвердил, запрос в эмулятор, итог не получен.
- SUCCESS — эмулятор подтвердил, чаевые начислены (конечное).
- DECLINED — эмулятор отказал (конечное).
- CANCELED — отменён клиентом или системой (конечное).

9.2. Матрица переходов

Из	В	Условие
CREATED	PENDING	клиент подтвердил, верный confirm_token
PENDING	SUCCESS	эмулятор вернул h ∈ 0..6
PENDING	DECLINED	эмулятор вернул h = 7
PENDING	PENDING	эмулятор вернул h = 8 (таймаут)

CREATED	CANCELED	клиент отменил до подтверждения
PENDING	CANCELED	эмулятор вернул h = 9 (система)
SUCCESS	CANCELED	запрещено (CANCEL_NOT_ALLOWED)

9.3. Формулы и правило эмулятора

tip_amount = round(base_amount × tip_pct / 100, 2) amount_total = base_amount + tip_amount

h = N mod 10, где N — числовая часть qr_id (qr_007 → 7)

SIGNATURE = HMAC-SHA256(SECRET, merchant_id|employee_id|receipt_id|nonce|exp)[:16]

h	Результат
0–6	SUCCESS
7	DECLINED
8	PENDING (таймаут)
9	CANCELED

9.4. Обработка ошибок

Ситуация	error_code	HTTP	retryable
QR signature не совпала	INVALID_QR	422	нет
QR истёк	QR_EXPIRED	410	да
nonce использован	QR_USED	409	нет
merchant BLOCKED	MERCHANT_BLOCKED	403	нет
receipt_id не совпадает	RECEIPT_MISMATCH	422	нет
tip_pct > 50	TIP_OUT_OF_RANGE	422	нет
amount_total > 1 200 000	AMOUNT_OUT_OF_RANGE	422	нет
эмулятор отказал (h=7)	PAYMENT_DECLINED	402	нет
эмулятор таймаут (h=8)	GATEWAY_TIMEOUT	202	да
эмулятор отменил (h=9)	PAYMENT_CANCELED	409	нет

Ситуация	error_code	HTTP	retryable
отмена из PENDING/SUCCESS	CANCEL_NOT_ALLOWED	409	нет

10. Отчётность

Правила: tips_total, avg_tip_pct, avg_tip_amount — только по SUCCESS; payments_count и success_rate — по всем платежам периода.

10.1. Отчёт по мерчанту

Endpoint: `GET /api/v1/reports/tips?merchant_id=m_001&from=2025-11-01&to=2025-11-30`

Метрика	Формула	Пример
tips_total	sum(tip) по SUCCESS	414.85
payments_count	count(payments)	6
avg_tip_pct	avg(tip / amount × 100)	9.0
success_rate	count(SUCCESS) / count(*)	0.83

10.2. Отчёт по официанту

Endpoint: `GET /api/v1/reports/tips?employee_id=e_001&from=…&to=…`

Метрика	Формула	Пример (e_001)
tips_total	sum(tip) SUCCESS официанта	83.60
payments_count	count(payments) официанта	2
avg_tip_amount	tips_total / count(SUCCESS)	41.80
top_day	день с макс. суммой чаевых	2025-11-03

10.3. Отчёт по дням

Endpoint: `GET /api/v1/reports/tips/daily?merchant_id=m_001&from=…&to=…`

Метрика	Формула
date	Дата
tips_total	sum(tip) за день (SUCCESS)
payments_count	count за день
avg_tip_pct	avg(tip / amount × 100) за день (SUCCESS)

11. Спецификация API

OpenAPI 3.0.3 — основные схемы и эндпоинты (фрагмент).

```
openapi: 3.0.3
info:
  title: Tips Platform   version: "1.0.0"
description: Онлайн-чаевые для сети «Вкус-Сервис»
servers:   - url: http://localhost:8083

components:
schemas:
    QRPayload:
      type: object       required: [merchant_id, employee_id, receipt_id,
nonce, exp, signature]       properties:
        merchant_id: {type: string, pattern: '^m_[0-9]{3}$'}
employee_id: {type: string, pattern: '^e_[0-9]{3}$'}
receipt_id: {type: string, pattern: '^r_[0-9]{4}$'}
nonce: {type: string, pattern: '^[a-f0-9]{6}$'}         exp:
{type: string, format: date-time}         signature: {type:
string, pattern: '^[a-f0-9]{16}$'}

    Payment:
      type: object       required: [payment_id, status,
amount_total, tip_amount]       properties:
        payment_id: {type: string, pattern: '^p_[0-9]{4}$'}         status: {type:
string, enum: [CREATED, PENDING, SUCCESS, DECLINED, CANCELED]}         amount_total:
{type: number, format: double}         tip_amount: {type: number, format: double}
confirm_token: {type: string}         error_code: {type: string, nullable: true}

    Error:
      type: object       required:
[error_code, message]
properties:
        error_code: {type: string}
message: {type: string}
retryable: {type: boolean}

paths:
/api/v1/qr/{qr_id}:
get:
      summary: Получить данные QR для формы чаевых
responses:
        '200': {description: QR действителен}
        '403': {description: MERCHANT_BLOCKED}
        '409': {description: QR_USED}
        '410': {description: QR_EXPIRED}
        '422': {description: INVALID_QR}

  /api/v1/tips:
post:
      summary: Создать платёж с чаевыми
responses:
        '201': {description: Платёж создан (CREATED)}
        '200': {description: Повтор idempotency_key}

  /api/v1/payments/{id}/confirm:
post:
      summary: Подтвердить платёж
responses:
        '200': {description: SUCCESS}
        '202': {description: PENDING (GATEWAY_TIMEOUT)}
        '402': {description: PAYMENT_DECLINED}

  /api/v1/payments/{id}/cancel:
post:
      summary: Отменить платёж (только из CREATED)
responses:
        '200': {description: CANCELED}
        '409': {description: CANCEL_NOT_ALLOWED}

  /api/v1/reports/tips:     get:       summary:
Отчёт по чаевым (мерчант / официант)

  /api/v1/reports/tips/daily:
get:
      summary: Отчёт по дням
```

12. Информационная безопасность

12.1. Модель угроз

ID	Угроза	Актив	Вер.	Ущерб
U-1	Подделка QR	деньги клиента	средняя	высокий
U-2	Replay QR	деньги клиента	средняя	средний
U-3	Кража реквизитов официанта	реквизиты выплат	низкая	высокий
U-4	Двойное списание	деньги клиента	средняя	средний
U-5	Манипуляции официанта с суммой	деньги клиента	средняя	средний
U-6	Отсутствие аудита	разбор споров	средняя	средний

12.2. Меры защиты

ID	Угроза	Мера	Ост. риск
U-1	подделка QR	HMAC-SHA256 + секрет	низкий
U-2	replay	nonce + TTL 24 ч	низкий
U-3	кража реквизитов	только маска ****NNNN	низкий
U-4	двойное списание	idempotency_key + одноразовый nonce	низкий
U-5	манипуляции суммой	QR ↔ receipt_id, tip ≤ 50%, total ≤ 1.2M	средний
U-6	нет аудита	журнал 8 полей, неизменяемый	низкий

12.3. Защита данных и секреты

- Нельзя хранить: PAN / CVV (обрабатывает только платёжный провайдер); полные реквизитыофицианта.
- Можно хранить: payment_details как ****NNNN; signature (результат, не ключ); nonce.
- HMAC-ключ: переменная окружения / секрет-хранилище, не в коде и Git. Ротация раз в 90 дней +при подозрении на утечку.
- Сравнение подписей — безопасной функцией (constant-time).

13. Приложение

13.1. Схема системы

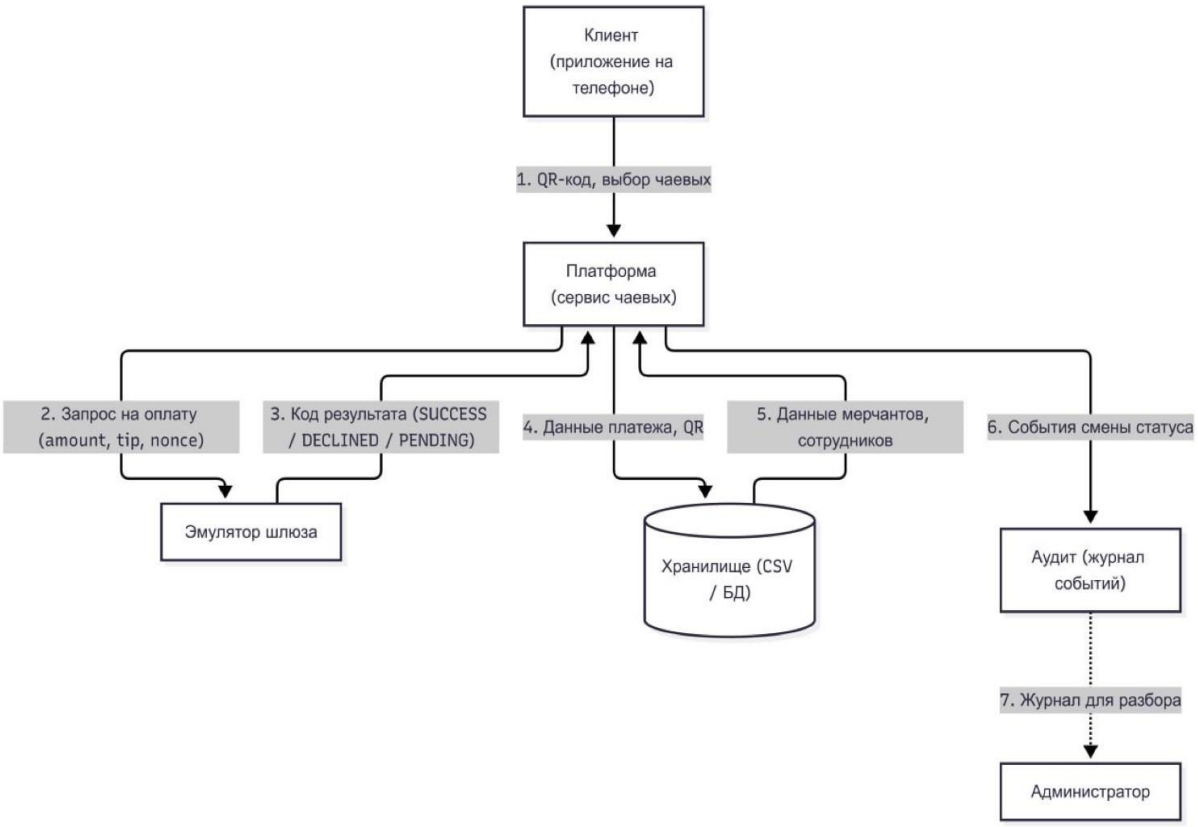


Рисунок 2. Схема системы: Клиент → Платформа → Эмулятор / Хранилище / Аудит

13.2. Схема состояний платежа

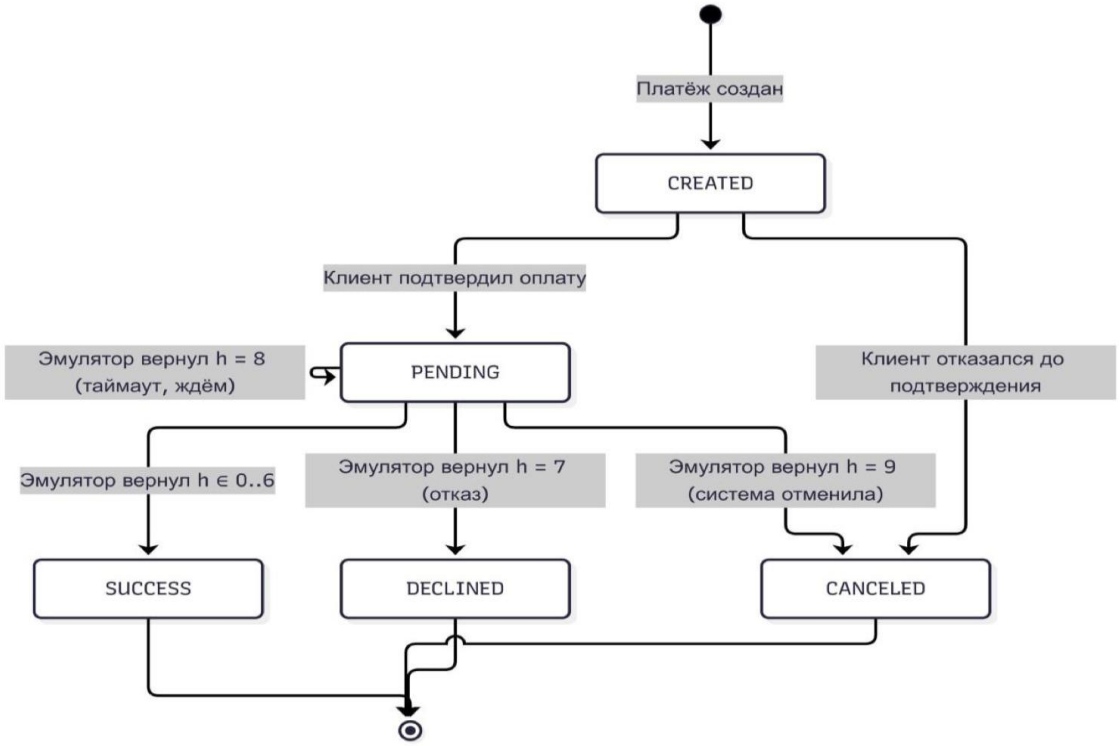


Рисунок 3. Диаграмма состояний платежа (CREATED → PENDING → SUCCESS/DECLINED/CANCELED)

13.3. Матрица «ситуация → код → HTTP → retryable»

Полная матрица приведена в разделе 9.4 (Обработка ошибок).

13.4. Журнал промптов (фрагмент)

Дата	Цель	Результат	Исп.
2026-11-01	Разобрать логику эмулятора	Python-скрипт для анализа 100 строк	да
2026-11-02	Исправить парсинг CSV в C++	Функции `split()` и `money_csv()`	перераб.
2026-11-03	Реализовать защиту P-2	Идея проверки `payment_id` в `payments_out.csv` перед оплатой	да
2026-11-04	Починить HMAC в C++	Объяснение про сырые байты и стандарт RFC 2104	перераб.
2026-11-05	Найти логическую дыру в P-3	Понял, что лимит в 2 млн недостижим, изменил на 1.2 млн	да
2026-11-06	Сгенерировать ER-модель	Код Mermaid	да
2026-11-08	Дополнить API-схему	YAML-фрагмент	Перераб.

13.5. Матрица вклада

Участник	Файлы / задачи	Часы
Базов Я.	Системный аналитик: `02_roles.md`, `02_правила.md`, `03_data_dictionary.md`, `08_errors.md` (ER-модель), `05_api_schemas.yaml`, настройка Git-репозитория (ветки, коммиты) и синхронизация документации	16
Ефимов М.Л	Разработчик: `code.cpp` (реализация эмулятора, HMAC-SHA256, идемпотентности), отладка C++ кода и подготовка тестовых данных	12
Имаев Т.Ю	Презентация и защита: `05_презентация.md`, `06_приложение.md` (сборка PDF), подготовка слайдов и защита проекта перед жюри	10

13.6. Артефакты

- Git-ветка: team_6_case_3
- Исходный код прототипа включён в раздел 14.
- Диаграммы ER, состояний и системы — выше.
- Данные CSV не изменялись (контроль целостности).
- Критерии приёмки по всем разделам отмечены выполненными.

13.7. Тесты

ID	Сценарий	Ввод	Ожидаемое	Факт
Т3-1	Успешная оплата	3 → qr_001 → 1 (5%).	подпись проверена, эмулятор SUCCESS (h = 1), платёж записан.	«[OK] Подпись QR проверена (HMAC-SHA256)», чек 1000.00 руб., «[Эмулятор] SUCCESS: Оплата прошла успешно.», «Платёж p_001 обновлён: SUCCESS». В payments_out.csv запись p_001,m_001,e_001,1000.00,50.00,SUCCESS.
Т3-2	Отказ банка	3 → qr_007 → 1 (5%)	DECLINED (h = 7).	подпись проверена, «[Эмулятор] DECLINED: банк отклонил.», «Платёж p_007 обновлён: DECLINED». В файле p_007,m_003,e_007,1000.00,50.00,DECLINED.
Т3-3	Неверная подпись Qr	1 → m_001 → 3 → e_001 → r_2001 → 1500 (создаётся qr_101); ещё раз 3 → e_001 → r_2002 → 800 (создаётся qr_102). В qr_links_generated.csv изменить последний символ подписи у qr_102. Затем 3 → qr_102.	INVALID_QR, платёж не обрабатывается.	«Ошибка: Неверная подпись QR! (INVALID_QR)». Запись p_102 осталась в статусе UNPAID. • Пройден: да.
Т3-3а	Генерация QR и оплата по нему	3 → qr_101 → 1 (5%).	чек берётся из созданной записи, SUCCESS (h = 1).	при создании: ID qr_101, сумма 1500.00 руб., истекает 2025-12-31T23:59:00Z, платёж p_101 со статусом UNPAID. При оплате: чек 1500.00, SUCCESS, запись p_101,m_001,e_001,1500.00,75.00,SUCCESS.
Т3-4	Повторная оплата (P-2)	после Т3-1 снова 3 → qr_001.	новый платёж не создаётся.	«[P-2] Платёж по этому QR уже был обработан ранее! Возврат существующего платежа. Новый не создаём.» В payments_out.csv одна запись p_001.
Т3-5	Отмена платежа эмулятором	3 → qr_009 → 1 (5%).	CANCELED (h = 9).	«[Эмулятор] CANCELED.», «Платёж p_009 обновлён: CANCELED». Отмена клиентом до подтверждения в прототипе не реализована.
Т3-6	Заблокированный мерчант	Ввод: 1 → m_019	MERCHANT_BLOCKED.	«Вход выполнен: Вкус-Сервис Ленинский (статус: BLOCKED)», «Ваш мерчант в статусе BLOCKED. (MERCHANT_BLOCKED)».
Т3-7	Чаевые больше 50%	3 → qr_002 → 4 → 60.	TIP_OUT_OF_RANGE.	«Введите процент (0-50): Ошибка: TIP_OUT_OF_RANGE», платёж не записан.
Т3-8	Таймаут эмулятора	3 → qr_008 → 1 (5%).	PENDING (h = 8).	«[Эмулятор] PENDING (timeout).», «Платёж p_008 обновлён: PENDING».

14. Исходный код прототипа (C++)

Консольный прототип на C++. Реализованы: HMAC-SHA256, проверка QR (Q-1…Q-5), генерация QR, эмулятор шлюза по h = qr_num % 10, идемпотентность (P-2), отчёты по мерчанту и официанту, маскирование реквизитов.

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <cstdio>
#include <cstdint>
#include <cstring>
#include <iomanip>

using namespace std;

// === Исходные файлы ===
const string QR_FILE          = "qr_links.csv";
const string PAYMENTS_FILE    = "payments.csv";
const string QR_GEN_FILE      = "qr_links_generated.csv";
const string PAYMENTS_OUT_FILE = "payments_out.csv";
const string SECRET_KEY       = "demo-secret-k3";

// === Ограничения ===
const double BASE_AMOUNT     = 1000.0;     // fallback для старых QR без
UNPAID
const double MAX_CHECK_AMOUNT = 1000000.0;   // верхняя граница чека при
создании QR
const double MAX_TOTAL_AMOUNT = 1200000.0;   // P-3: верхняя граница суммы
с чаевыми

//
================================================================
// РЕАЛИЗАЦИЯ SHA-256 И HMAC (нужно для проверки подписи QR - Q-1)
//
================================================================
namespace SHA256 {
   uint32_t rotr(uint32_t x, int n) { return (x >> n) | (x << (32 - n)); }

   void transform(uint32_t state[8], const uint8_t block[64]) {
      uint32_t w[64];
      const uint32_t K[64] = {
         0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1,
0x923f82a4, 0xab1c5ed5,
         0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74,
0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
         0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa,
0x5cb0a9dc, 0x76f988da,
         0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147,
0x06ca6351, 0x14292967,
         0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354,
0x766a0abb, 0x81c2c92e, 0x92722c85,
         0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819,
0xd6990624, 0xf40e3585, 0x106aa070,
         0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3,
0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
         0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb,
0xbef9a3f7, 0xc67178f2
      };
```

```
        for (int i = 0; i < 16; ++i)
            w[i] = ((uint32_t)block[i*4] << 24) | ((uint32_t)block[i*4+1] << 16) |
((uint32_t)block[i*4+2] << 8) | ((uint32_t)block[i*4+3]);
        for (int i = 16; i < 64; ++i) {
            uint32_t s0 = rotr(w[i-15], 7) ^ rotr(w[i-15], 18) ^ (w[i-15] >> 3);
            uint32_t s1 = rotr(w[i-2], 17) ^ rotr(w[i-2], 19) ^ (w[i-2] >> 10);
            w[i] = w[i-16] + s0 + w[i-7] + s1;
        }

        uint32_t a = state[0], b = state[1], c = state[2], d = state[3];
        uint32_t e = state[4], f = state[5], g = state[6], h = state[7];
        for (int i = 0; i < 64; ++i) {
            uint32_t S1 = rotr(e, 6) ^ rotr(e, 11) ^ rotr(e, 25);
            uint32_t ch = (e & f) ^ (~e & g);
            uint32_t temp1 = h + S1 + ch + K[i] + w[i];
            uint32_t S0 = rotr(a, 2) ^ rotr(a, 13) ^ rotr(a, 22);
            uint32_t maj = (a & b) ^ (a & c) ^ (b & c);
            uint32_t temp2 = S0 + maj;
            h = g; g = f; f = e; e = d + temp1;
            d = c; c = b; b = a; a = temp1 + temp2;
        }
        state[0] += a; state[1] += b; state[2] += c; state[3] += d;
        state[4] += e; state[5] += f; state[6] += g; state[7] += h;
    }

    void hash_raw(const string& input, uint8_t out[32]) {
        uint32_t state[8] = {0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19};
        uint64_t bit_len = input.length() * 8;

        string msg = input;
        msg += (char)0x80;
        while (msg.length() % 64 != 56) msg += (char)0x00;
        for (int i = 7; i >= 0; --i) msg += (char)((bit_len >> (i * 8)) & 0xFF);

        for (size_t i = 0; i < msg.length(); i += 64) {
            transform(state, (const uint8_t*)&msg[i]);
        }

        for (int i = 0; i < 8; ++i) {
            out[i*4]   = (state[i] >> 24) & 0xFF;
            out[i*4+1] = (state[i] >> 16) & 0xFF;
            out[i*4+2] = (state[i] >> 8) & 0xFF;
            out[i*4+3] = state[i] & 0xFF;
        }
    }

    string hmac_sha256(const string& key, const string& msg) {
        uint8_t o_key_pad[64], i_key_pad[64];
        memset(o_key_pad, 0x5c, 64);
        memset(i_key_pad, 0x36, 64);

        for (size_t i = 0; i < key.length() && i < 64; ++i) {
            o_key_pad[i] ^= key[i];
            i_key_pad[i] ^= key[i];
        }

        string inner_in = string((char*)i_key_pad, 64) + msg;
```

```
        uint8_t inner_hash[32];
        hash_raw(inner_in, inner_hash);

        string outer_in = string((char*)o_key_pad, 64) + string((char*)inner_hash, 32);
        uint8_t outer_hash[32];
        hash_raw(outer_in, outer_hash);

        stringstream ss;
        for (int i = 0; i < 32; ++i) {
            ss << hex << setw(2) << setfill('0') << (int)outer_hash[i];
        }
        return ss.str();
    }
}

// === Вспомогательные функции (ВОЗВРАЩЕНЫ!) ===
vector<string> split(const string& line) {
    vector<string> result;
    stringstream ss(line);
    string item;
    while (getline(ss, item, ',')) result.push_back(item);
    return result;
}

string money(double v) {
    char buf[32];
    snprintf(buf, sizeof(buf), "%.2f", v);
    return string(buf);
}

// Формат денег для CSV (точка вместо запятой)
string money_csv(double v) {
    char buf[32];
    snprintf(buf, sizeof(buf), "%.2f", v);
    string s(buf);
    for (auto& c : s) if (c == ',') c = '.';
    return s;
}

bool file_empty_or_missing(string filename) {
    ifstream f(filename);
    if (!f.is_open()) return true;
    f.seekg(0, ios::end);
    bool empty = (f.tellg() == 0);
    f.close();
    return empty;
}

string generate_hex(int len) {
    string hex = "0123456789abcdef";
    string res = "";
    for (int i = 0; i < len; i++) res += hex[rand() % 16];
    return res;
}

// Обновляет запись платежа по payment_id. Если нет — добавляет.
bool update_or_append_payment(const string& payment_id,
                              const string& m_id, const string& e_id,
```

```
                    double amount, double tip,
                    const string& status, const string& timestamp) {
    ifstream in(PAYMENTS_OUT_FILE);
    if (in.is_open()) {
        vector<string> lines;
        string line;
        bool found = false;
        while (getline(in, line)) {
            vector<string> parts = split(line);
            if (parts.size() >= 1 && parts[0] == payment_id) {
                stringstream ss;
                ss << payment_id << "," << m_id << "," << e_id << ","
                   << money_csv(amount) << "," << money_csv(tip) << ","
                   << status << "," << timestamp;
                line = ss.str();
                found = true;
            }
            lines.push_back(line);
        }
        in.close();
        if (found) {
            ofstream out(PAYMENTS_OUT_FILE);
            if (!out.is_open()) return false;
            for (auto& l : lines) out << l << "\n";
            out.close();
            return true;
        }
    }

    bool need_header = file_empty_or_missing(PAYMENTS_OUT_FILE);
    ofstream out(PAYMENTS_OUT_FILE, ios::app);
    if (!out.is_open()) return false;
    if (need_header)
        out << "payment_id,merchant_id,employee_id,amount,tip,status,timestamp"
<< endl;
    out << payment_id << "," << m_id << "," << e_id << ","
        << money_csv(amount) << "," << money_csv(tip) << ","
        << status << "," << timestamp << endl;
    out.close();
    return true;
}

// === Поиск по файлам ===
bool find_merchant(string m_id, string& name, string& status) {
    ifstream f("merchants.csv");
    string line;
    getline(f, line);
    while (getline(f, line)) {
        vector<string> parts = split(line);
        if (parts.size() >= 3 && parts[0] == m_id) {
            name = parts[1]; status = parts[2];
            return true;
        }
    }
    return false;
}

bool find_officer(string e_id, string& m_id, string& name, string& details) {
```

```
    ifstream f("employees.csv");
    string line;
    getline(f, line);
    while (getline(f, line)) {
        vector<string> parts = split(line);
        if (parts.size() >= 4 && parts[0] == e_id) {
            m_id = parts[1]; name = parts[2]; details = parts[3];
            return true;
        }
    }
    return false;
}

string get_merchant_status(string m_id) {
    string name, status;
    if (find_merchant(m_id, name, status)) return status;
    return "";
}

bool find_qr_in_file(string filename, string target, string& m_id, string& e_id,
                     string& r_id, string& nonce, string& exp, string& sig) {
    ifstream f(filename);
    if (!f.is_open()) return false;
    string line;
    getline(f, line);
    while (getline(f, line)) {
        vector<string> parts = split(line);
        if (parts.size() >= 7 && parts[0] == target) {
            m_id = parts[1]; e_id = parts[2]; r_id = parts[3];
            nonce = parts[4]; exp = parts[5]; sig = parts[6];
            f.close();
            return true;
        }
    }
    f.close();
    return false;
}

bool find_qr(string target, string& m_id, string& e_id, string& r_id,
             string& nonce, string& exp, string& sig) {
    if (find_qr_in_file(QR_FILE, target, m_id, e_id, r_id, nonce, exp, sig)) return true;
    if (find_qr_in_file(QR_GEN_FILE, target, m_id, e_id, r_id, nonce, exp, sig)) return
true;
    return false;
}

// === Генерация QR ===
int find_max_qr_number() {
    int max_num = 0;
    string files[] = {QR_FILE, QR_GEN_FILE};
    for (int fi = 0; fi < 2; fi++) {
        ifstream f(files[fi]);
        if (!f.is_open()) continue;
        string line;
        getline(f, line);
        while (getline(f, line)) {
            vector<string> parts = split(line);
            if (parts.size() >= 1 && parts[0].length() > 3) {
```

```
            int num = atoi(parts[0].substr(3).c_str());
            if (num > max_num) max_num = num;
          }
        }
        f.close();
    }
    return max_num;
}

string format_qr_id(int n) {
    string s = to_string(n);
    while (s.length() < 3) s = "0" + s;
    return "qr_" + s;
}

bool receipt_exists(string m_id, string r_id) {
    string files[] = {QR_FILE, QR_GEN_FILE};
    for (int fi = 0; fi < 2; fi++) {
        ifstream f(files[fi]);
        if (!f.is_open()) continue;
        string line;
        getline(f, line);
        while (getline(f, line)) {
            vector<string> parts = split(line);
            if (parts.size() >= 4 && parts[1] == m_id && parts[3] == r_id) {
                f.close();
                return true;
            }
        }
        f.close();
    }
    return false;
}

void generate_qr(string m_id, string e_id) {
    string status = get_merchant_status(m_id);
    if (status != "ACTIVE") {
        cout << "Ошибка: мерчант в статусе " << status
             << " — QR не создаётся (MERCHANT_BLOCKED)" << endl;
        return;
    }
    string r_id;
    cout << "Введите номер чека (например, r_2001): ";
    cin >> r_id;

    if (receipt_exists(m_id, r_id)) {
        cout << "Ошибка: для чека " << r_id << " уже есть QR." << endl;
        return;
    }

    double base_amount = 0.0;
    while (true) {
        cout << "Введите сумму чека в рублях (например, 1500 или 1500.50): ";
        string amt_str;
        cin >> amt_str;
        for (auto& ch : amt_str) if (ch == ',') ch = '.';
        char* endp = nullptr;
        base_amount = strtod(amt_str.c_str(), &endp);
```

```
        if (endp == amt_str.c_str() || *endp != '\0' || base_amount <= 0.0 ||
base_amount > MAX_CHECK_AMOUNT) {
            cout << "Ошибка: введите число от 0.01 до " <<
money(MAX_CHECK_AMOUNT) << " руб." << endl;
            continue;
        }
        break;
    }

    int next_num = find_max_qr_number() + 1;
    string qr_id = format_qr_id(next_num);
    string nonce = generate_hex(6);
    string exp   = "2025-12-31T23:59:00Z";

    string payload = m_id + "|" + e_id + "|" + r_id + "|" + nonce + "|" + exp;
    string full_hash = SHA256::hmac_sha256(SECRET_KEY, payload);
    string sig = full_hash.substr(0, 16);

    bool need_header = file_empty_or_missing(QR_GEN_FILE);
    ofstream f(QR_GEN_FILE, ios::app);
    if (!f.is_open()) {
        cout << "Ошибка записи в " << QR_GEN_FILE << endl;
        return;
    }
    if (need_header)
        f << "qr_id,merchant_id,employee_id,receipt_id,nonce,exp,signature" << endl;
    f << qr_id << "," << m_id << "," << e_id << "," << r_id << ","
      << nonce << "," << exp << "," << sig << endl;
    f.close();

    // Отправляем QR в файл оплаты со статусом UNPAID
    string payment_id = "p_" + qr_id.substr(3);
    bool unpaid_ok = update_or_append_payment(payment_id, m_id, e_id,
base_amount, 0.0, "UNPAID", "0000-00-00T00:00:00Z");

    cout << "\n=== QR УСПЕШНО СОЗДАН (записан в " << QR_GEN_FILE << ") ==="
<< endl;
    cout << "ID:       " << qr_id << endl;
    cout << "Мерчант:  " << m_id << endl;
    cout << "Официант: " << e_id << endl;
    cout << "Чек:      " << r_id << endl;
    cout << "Сумма:    " << money(base_amount) << " руб." << endl;
    cout << "Истекает: " << exp << endl;
    if (unpaid_ok) {
        cout << "Платёж " << payment_id << " создан со статусом UNPAID в " <<
PAYMENTS_OUT_FILE << endl;
    } else {
        cout << "Предупреждение: не удалось создать запись UNPAID в " <<
PAYMENTS_OUT_FILE << endl;
    }
    cout << "Сообщите клиенту ID: " << qr_id << endl;
}

// === Отчёты ===
void show_merchant_report(string m_id) {
    double tips_total = 0;
    int payments_count = 0, success_count = 0, pct_count = 0;
    double pct_sum = 0;
```

```
    string files[] = {PAYMENTS_FILE, PAYMENTS_OUT_FILE};
    for (int fi = 0; fi < 2; fi++) {
        ifstream f(files[fi]);
        if (!f.is_open()) continue;
        string line;
        getline(f, line);
        while (getline(f, line)) {
            vector<string> parts = split(line);
            if (parts.size() < 6 || parts[1] != m_id) continue;
            if (parts[5] == "UNPAID") continue;
            payments_count++;
            if (parts[5] == "SUCCESS") {
                double tip = atof(parts[4].c_str()), amount = atof(parts[3].c_str());
                tips_total += tip; success_count++;
                if (amount > 0) { pct_sum += (tip / amount) * 100.0; pct_count++; }
            }
        }
        f.close();
    }
    double avg_pct = pct_count > 0 ? pct_sum / pct_count : 0;
    double success_rate = payments_count > 0 ? (double)success_count /
payments_count : 0;
    cout << "\n=== ОТЧЁТ ПО ЗАВЕДЕНИЮ " << m_id << " ===" << endl;
    cout << "Всего платежей:         " << payments_count << endl;
    cout << "Успешных:               " << success_count << endl;
    cout << "Сумма чаевых (SUCCESS):  " << money(tips_total) << " руб." << endl;
    cout << "Средний процент чаевых:  " << money(avg_pct) << "%" << endl;
    cout << "Доля успешных:          " << money(success_rate) << endl;
}

void show_merchant_officers(string m_id) {
    cout << "\n=== ОФИЦИАНТЫ ЗАВЕДЕНИЯ " << m_id << " ===" << endl;
    ifstream f("employees.csv");
    string line;
    getline(f, line);
    int count = 0;
    while (getline(f, line)) {
        vector<string> parts = split(line);
        if (parts.size() >= 4 && parts[1] == m_id) {
            cout << parts[0] << " | " << parts[2] << " | реквизиты: " << parts[3] << endl;
            count++;
        }
    }
    if (count == 0) cout << "(нет официантов)" << endl;
}

void show_officer_tips(string e_id) {
    double tips_total = 0;
    int success_count = 0, total_count = 0;
    string files[] = {PAYMENTS_FILE, PAYMENTS_OUT_FILE};
    for (int fi = 0; fi < 2; fi++) {
        ifstream f(files[fi]);
        if (!f.is_open()) continue;
        string line;
        getline(f, line);
        while (getline(f, line)) {
            vector<string> parts = split(line);
            if (parts.size() < 6 || parts[2] != e_id) continue;
```

```
            if (parts[5] == "UNPAID") continue;
            total_count++;
            if (parts[5] == "SUCCESS") {
                tips_total += atof(parts[4].c_str()); success_count++;
            }
        }
        f.close();
    }
    double avg = success_count > 0 ? tips_total / success_count : 0;
    cout << "\n=== ВАШИ ЧАЕВЫЕ (" << e_id << ") ===" << endl;
    cout << "Успешных платежей:       " << success_count << " из " << total_count <<
endl;
    cout << "Сумма чаевых:            " << money(tips_total) << " руб." << endl;
    cout << "Средние чаевые за платёж: " << money(avg) << " руб." << endl;
}

// === Сессии ===
void merchant_session(string m_id, string m_name, string m_status) {
    if (m_status != "ACTIVE") {
        cout << "\nВаш мерчант в статусе " << m_status << ". (MERCHANT_BLOCKED)"
<< endl;
        return;
    }
    while (true) {
        cout << "\n--- МЕНЮ МЕРЧАНТА: " << m_name << " (" << m_id << ") ---" <<
endl;
        cout << "1. Отчёт по заведению\n2. Список официантов\n3. Сгенерировать
QR на чек\n0. Выйти\nВыбор: ";
        int c; cin >> c;
        if (c == 1) show_merchant_report(m_id);
        else if (c == 2) show_merchant_officers(m_id);
        else if (c == 3) {
            string e_id;
            cout << "Введите ID официанта (например, e_001): ";
            cin >> e_id;
            string em_id, ename, edet;
            if (!find_officer(e_id, em_id, ename, edet)) cout << "Ошибка: официант не
найден." << endl;
            else if (em_id != m_id) cout << "Ошибка: этот официант не работает в
вашем заведении." << endl;
            else generate_qr(m_id, e_id);
        }
        else if (c == 0) break;
    }
}

void officer_session(string e_id, string e_name, string e_m_id) {
    string m_status = get_merchant_status(e_m_id);
    if (m_status != "ACTIVE") {
        cout << "\nМерчант " << e_m_id << " в статусе " << m_status << " — QR
недоступен." << endl;
        return;
    }
    while (true) {
        cout << "\n--- МЕНЮ ОФИЦИАНТА: " << e_name << " (" << e_id << ") ---" <<
endl;
        cout << "1. Сгенерировать QR на чек\n2. Мои чаевые\n0. Выйти\nВыбор: ";
        int c; cin >> c;
```

```
        if (c == 1) generate_qr(e_m_id, e_id);
        else if (c == 2) show_officer_tips(e_id);
        else if (c == 0) break;
    }
}

// === Оплата (Клиент) ===
void client_pay() {
    cout << "\n--- ОПЛАТА ЧАЕВЫХ (КЛИЕНТ) ---" << endl;
    cout << "Введите ID QR-кода с чека (например, qr_001): ";
    string target; cin >> target;

    string m_id, e_id, r_id, nonce, exp, sig;
    if (!find_qr(target, m_id, e_id, r_id, nonce, exp, sig)) {
        cout << "Ошибка: QR не найден (INVALID_QR)" << endl;
        return;
    }

    string now = "2025-11-01T12:00:00Z";
    if (exp < now) {
        cout << "Ошибка: QR истёк (QR_EXPIRED)" << endl;
        return;
    }

    string payload = m_id + "|" + e_id + "|" + r_id + "|" + nonce + "|" + exp;
    string full_hash = SHA256::hmac_sha256(SECRET_KEY, payload);
    string expected_sig = full_hash.substr(0, 16);
    if (expected_sig != sig) {
        cout << "Ошибка: Неверная подпись QR! (INVALID_QR)" << endl;
        return;
    }
    cout << "[OK] Подпись QR проверена (HMAC-SHA256)." << endl;

    string payment_id = "p_" + target.substr(3);
    bool already_paid  = false;
    bool found_unpaid  = false;
    double stored_amount = 0.0;

    ifstream check_out(PAYMENTS_OUT_FILE);
    if (check_out.is_open()) {
        string line;
        getline(check_out, line);
        while (getline(check_out, line)) {
            vector<string> parts = split(line);
            if (parts.size() >= 6 && parts[0] == payment_id) {
                if (parts[5] != "UNPAID") {
                    already_paid = true;
                } else {
                    found_unpaid  = true;
                    stored_amount = atof(parts[3].c_str());
                }
            }
        }
        check_out.close();
    }

    if (already_paid) {
        cout << "\n[P-2] Платёж по этому QR уже был обработан ранее!" << endl;
```

```
        cout << "Возврат существующего платежа. Новый не создаём." << endl;
        return;
    }

    double base_amount = found_unpaid ? stored_amount : BASE_AMOUNT;

    string m_status = get_merchant_status(m_id);
    if (m_status != "ACTIVE") {
        cout << "Ошибка: заведение недоступно (MERCHANT_BLOCKED, " <<
m_status << ")" << endl;
        return;
    }

    cout << "\nЗаведение: " << m_id << " (официант " << e_id << ")" << endl;
    cout << "Чек: " << money(base_amount) << " руб." << endl;
    cout << "\nВыберите чаевые:\n1. 5%\n2. 10%\n3. 20%\n4. Своя сумма\n5. Без
чаевых\nВыбор: ";
    int c; cin >> c;
    double tip_pct = 0;
    if      (c == 1) tip_pct = 5;
    else if (c == 2) tip_pct = 10;
    else if (c == 3) tip_pct = 20;
    else if (c == 5) tip_pct = 0;
    else if (c == 4) {
        cout << "Введите процент (0-50): "; cin >> tip_pct;
        if (tip_pct < 0 || tip_pct > 50) { cout << "Ошибка: TIP_OUT_OF_RANGE" <<
endl; return; }
    } else { cout << "Неверный выбор" << endl; return; }

    double tip = base_amount * tip_pct / 100.0;
    double total = base_amount + tip;

    if (total > MAX_TOTAL_AMOUNT) {
        cout << "Ошибка: AMOUNT_OUT_OF_RANGE" << endl;
        return;
    }

    int qr_num = atoi(target.substr(3).c_str());
    int h = qr_num % 10;
    string status;
    if (h <= 6)      { status = "SUCCESS";  cout << "\n[Эмулятор] SUCCESS: Оплата
прошла успешно." << endl; }
    else if (h == 7) { status = "DECLINED"; cout << "\n[Эмулятор] DECLINED: банк
отклонил." << endl; }
    else if (h == 8) { status = "PENDING";  cout << "\n[Эмулятор] PENDING (timeout)."
<< endl; }
    else             { status = "CANCELED"; cout << "\n[Эмулятор] CANCELED." << endl; }

    if (update_or_append_payment(payment_id, m_id, e_id, base_amount, tip,
status, "2025-11-01T18:00:00Z")) {
        cout << "Платёж " << payment_id << " обновлён: " << status << " (файл " <<
PAYMENTS_OUT_FILE << ")" << endl;
    } else {
        cout << "Ошибка: не удалось обновить/сохранить платёж." << endl;
    }
}

// === Вход ===
```

```
void login_merchant() {
    string m_id;
    cout << "Введите merchant_id (например, m_001): "; cin >> m_id;
    string name, status;
    if (!find_merchant(m_id, name, status)) { cout << "Мерчант не найден." << endl;
return; }
    cout << "\nВход выполнен: " << name << " (статус: " << status << ")" << endl;
    merchant_session(m_id, name, status);
}

void login_officer() {
    string e_id;
    cout << "Введите employee_id (например, e_001): "; cin >> e_id;
    string m_id, name, details;
    if (!find_officer(e_id, m_id, name, details)) { cout << "Официант не найден." <<
endl; return; }
    cout << "\nВход выполнен: " << name << " (мерчант: " << m_id << ")" << endl;
    officer_session(e_id, name, m_id);
}

// === main ===
int main() {
    system("chcp 1251 > nul");
    setlocale(LC_ALL, "ru_RU.UTF-8");
    srand(time(0));

    while (true) {
        cout << "\n======================================" << endl;
        cout << "  ПЛАТФОРМА ДЛЯ ОПЛАТЫ 'ВКУС-СЕРВИС'" << endl;
        cout << "======================================" << endl;
        cout << "1. Вход мерчанта\n2. Вход официанта\n3. Оплатить (клиент)\n0.
Выход\nВыберите: ";
        int c; cin >> c;
        if      (c == 1) login_merchant();
        else if (c == 2) login_officer();
        else if (c == 3) client_pay();
        else if (c == 0) break;
        else cout << "Неверный пункт." << endl;
    }
    return 0;
}
```

Реализованно в коде	Описано в проекте и API, в прототипе нет
Подпись HMAC-SHA256(Q-1)	Статусы CREATED/PENDING, confirm_token
Срок QR (Q-2,”Сегодня” зашито в коде)	Отмена платежа (FR-8,P-4)
Статус мерчанта ACTIVE(Q-4)	Журнал аудита (FR-9)
Чаевые <= 50% (Q-6), сумма <=1200000 (P-3)	Idempotency_key, Q-3 (nonce), Q-5 (receipt_id)
Эмулятор шлюза qr_num % 10	Отчёт по дням, rate limit, TTl 24 ч
Защита от повторной оплаты по qr_id (аналог P-2)	Ключ из getenv, constant-time сравнение